

MINOR PROJECT 3

```

1  -- RedFlag – Fraud Detection Submission
2  -- Student: B Harshitha
3
4
5
6  USE
7  redflag;
8
9  -- =====
10 -- PATTERN 1 · VELOCITY FRAUD
11 -- What I'm looking for: users with 30 or more transactions on one
12 -- calendar day.
13 -- Expected suspects: ~50
14 -- =====
15 SELECT
16     user_id,
17     DATE(txn_time) AS
18     attack_date, COUNT(*) AS
19     daily_txn_count
20 FROM transactions
21 GROUP BY user_id,
22 DATE(txn_time) HAVING
23 COUNT(*) >= 30
24 ORDER BY daily_txn_count
25 DESC;
26 -- My findings: 50 suspect user-days were flagged. Examples include
27 -- with 60 transactions on 2024-04-03 and user 14556 with 60
28 -- transactions
29 -- on 2024-05-28.
30
31 =====
32 -- PATTERN 2 · ROUND-AMOUNT CLUSTERING
33 -- What I'm looking for: users with 15 or more transactions at the
34 -- specified
35 -- exactly-round amounts.
36 -- Expected suspects: 25
37
38 SELECT
39     user_id,
40     COUNT(*) AS
41     round_txn_count FROM
42     transactions
43 WHERE amount IN (100, 200,
44 500, 1000, 2000, 5000,
45 10000)
46 GROUP BY user_id
47 HAVING COUNT(*) >= 15
48 ORDER BY round_txn_count
49 DESC;
50 -- My findings: 25 suspect users were flagged. Examples include
51 -- user 14535 with 30 round-amount transactions (first observed on 2024-01-01)
52 -- and
53 -- user 14535 with 30 (first observed on 2024-01-10).
54
55 =====
56 ==
57 -- PATTERN 3 · CARD TESTING
58 -- What I'm looking for: users with 30 or more transactions under
59 ₹10
60 -- on one calendar day.

```

```

51 -- Expected suspects: 20
52 -- =====
53
54 SELECT
55     user_id,
56     DATE(txn_time) AS attack_date,
57     COUNT(*) AS tiny_txn_count
58 FROM transactions
59 WHERE amount < 10
60 GROUP BY user_id, DATE(txn_time)
61 HAVING COUNT(*) >= 30
62 ORDER BY tiny_txn_count DESC;
63
64 -- My findings: 20 suspect user-days were flagged. Examples include
65 -- user 14556
66 -- with 60 tiny transactions on 2024-05-28 and user 14569 with 60 on
67 -- 2024-04-03.
68 -- =====
69 -- PATTERN 4 · FAILED-THEN-SUCCEEDED
70 -- What I'm looking for: users with 20 or more failed transactions that
71 -- are
72 -- followed within 2 minutes by a successful transaction of the same
73 -- amount.
74 -- Expected suspects: 25
75 -- =====
76
77 SELECT
78     f.user_id,
79     COUNT(*) AS failed_then_success_pairs
80 FROM transactions f
81 WHERE f.status = 'FAILED'
82     AND EXISTS (
83         SELECT 1
84         FROM transactions s
85         WHERE s.user_id = f.user_id
86             AND s.status = 'SUCCESS'
87             AND s.amount = f.amount
88             AND s.txn_time > f.txn_time
89             AND s.txn_time <= DATE_ADD(f.txn_time, INTERVAL 2 MINUTE)
90     )
91 GROUP BY f.user_id
92 HAVING COUNT(*) >= 20
93 ORDER BY failed_then_success_pairs DESC;
94
95 -- My findings: 25 suspect users were flagged. Examples include user
96 -- 14595
97 -- with 35 failed-then-success pairs, including a pair on 2024-05-25,
98 -- and user 14593 with 34 pairs, including pairs on 2024-04-19.
99 -- =====
100 -- PATTERN 5 · ODD-HOUR CONCENTRATION
101 -- What I'm looking for: users with at least 30 transactions where 80%
102 -- or
103 -- more occur during hours 2, 3, or 4.
104 -- Expected suspects: 20
105 -- =====

```

```

105     user_id,
106     COUNT(*) AS total_txns,
107     SUM(
108         CASE
109             WHEN HOUR(txn_time) BETWEEN 2 AND 4 THEN 1
110             ELSE 0
111         END
112     ) AS odd_hour_txns,
113     ROUND(
114         SUM(
115             CASE
116                 WHEN HOUR(txn_time) BETWEEN 2 AND 4 THEN 1
117                 ELSE 0
118             END
119         ) / COUNT(*),
120         4
121     ) AS odd_hour_ratio
122 FROM transactions
123 GROUP BY user_id
124 HAVING COUNT(*) >= 30
125     AND SUM(
126         CASE
127             WHEN HOUR(txn_time) BETWEEN 2 AND 4 THEN 1
128             ELSE 0
129         END
130     ) / COUNT(*) >= 0.80
131 ORDER BY odd_hour_ratio DESC;
132
133 -- My findings: 20 suspect users were flagged. Examples include user
134 -- 14606
135 -- with 49 of 52 transactions in the odd-hour window and user 14609 with
136 -- 45 of 48 transactions in the same window.
137
138 -- =====
139 -- PATTERN 6 · MULE ACCOUNTS
140 -- What I'm looking for: users with at least 5 instances where a CREDIT
141 -- through
142 -- NETBANKING is followed within 30 minutes by a DEBIT through UPI worth
143 -- at
144 -- least 70% of the credited amount.
145 -- Expected suspects: 30
146 -- =====
147
148 SELECT
149     c.user_id,
150     COUNT(*) AS credit_to_debit_instances
151 FROM transactions c
152 WHERE c.txn_type = 'CREDIT'
153     AND c.payment_mode = 'NETBANKING'
154     AND EXISTS (
155         SELECT 1
156         FROM transactions d
157         WHERE d.user_id = c.user_id
158             AND d.txn_type = 'DEBIT'
159             AND d.payment_mode = 'UPI'
160             AND d.txn_time > c.txn_time
161             AND d.txn_time <= DATE_ADD(c.txn_time, INTERVAL 30 MINUTE)

```

```

159         AND d.amount >= c.amount * 0.70
160     )
161 GROUP BY c.user_id
162 HAVING COUNT(*) >= 5
163 ORDER BY credit_to_debit_instances DESC;
164
165 -- My findings: 30 suspect users were flagged. Examples include user
166 -- 14621,
167 -- which has multiple qualifying credit-to-debit instances, including
168 -- one
169 -- beginning on 2024-01-18.
170
171 -- =====
172 -- PATTERN 7 · REFUND ABUSE
173 -- What I'm looking for: users with 20 or more total transactions and a
174 -- refund
175 -- ratio greater than 40%.
176 -- Expected suspects: 24-25
177 -- =====
178
179 SELECT
180     user_id,
181     COUNT(*) AS total_txns,
182     SUM(
183         CASE
184             WHEN txn_type = 'REFUND' THEN 1
185             ELSE 0
186         END
187     ) AS refund_txns,
188     ROUND(
189         SUM(
190             CASE
191                 WHEN txn_type = 'REFUND' THEN 1
192                 ELSE 0
193             END
194         ) / COUNT(*),
195         4
196     ) AS refund_ratio
197 FROM transactions
198 GROUP BY user_id
199 HAVING COUNT(*) >= 20
200     AND SUM(
201         CASE
202             WHEN txn_type = 'REFUND' THEN 1
203             ELSE 0
204         END
205     ) / COUNT(*) > 0.40
206 ORDER BY refund_ratio DESC;
207
208 -- My findings: 24 suspect users were flagged. Examples include user
209 -- 14662
210 -- with 25 refunds out of 39 transactions, with refund activity from
211 -- 2024-01-08 to 2024-06-03, and user 14670 with 32 refunds out of 50
212 -- transactions.
213
214 -- =====
215 -- PATTERN 8 · MERCHANT COLLUSION

```

```

213 -- account for more than 60% of the merchant's total transaction
214 value.
215 -- Expected suspects: 15 merchants
216 --
217 =====
218 WITH user_merchant_volume AS (
219     SELECT
220         merchant_id,
221         user_id,
222         SUM(amount) AS user_volume
223     FROM transactions
224     GROUP BY merchant_id, user_id
225 ),
226 ranked_users AS (
227     SELECT
228         merchant_id,
229         user_id,
230         user_volume,
231         ROW_NUMBER() OVER (
232             PARTITION BY merchant_id
233             ORDER BY user_volume DESC, user_id
234         ) AS user_rank
235     FROM user_merchant_volume
236 ),
237 merchant_totals AS (
238     SELECT
239         merchant_id,
240         SUM(amount) AS merchant_total
241     FROM transactions
242     GROUP BY merchant_id
243 ),
244 top_five AS (
245     SELECT
246         merchant_id,
247         SUM(user_volume) AS top5_volume
248     FROM ranked_users
249     WHERE user_rank <= 5
250     GROUP BY merchant_id
251 )
252 SELECT
253     t.merchant_id,
254     t.top5_volume,
255     m.merchant_total,
256     ROUND(t.top5_volume / m.merchant_total, 4) AS top5_volume_ratio
257 FROM top_five t
258 JOIN merchant_totals m
259     ON m.merchant_id = t.merchant_id
260 WHERE t.top5_volume / m.merchant_total > 0.60
261 ORDER BY top5_volume_ratio DESC;
262
263 -- My findings: 15 suspect merchants were flagged. The flagged
264 merchants are
265 -- merchant IDs 1 through 15; for example, merchant 12 has a top-five
266 volume
267 -- ratio of about 99.91%, and merchant 8 has about 99.87%.

```

```

267 -- PATTERN 9 · JUST-UNDER-THRESHOLD (STRUCTURING)
268 -- What I'm looking for: users with 10 or more transactions at exactly
269 -- Expected suspects: 20
270 -- =====
271
272 SELECT
273     user_id,
274     COUNT(*) AS threshold_txn_count
275 FROM transactions
276 WHERE amount = 9999.00
277 GROUP BY user_id
278 HAVING COUNT(*) >= 10
279 ORDER BY threshold_txn_count DESC;
280
281 -- My findings: 20 suspect users were flagged. Examples include user
282 -- with 25 transactions at ₹9,999, with activity from 2024-01-13 to
283 -- 2024-06-18, and user 14680 with 25 such transactions.
284
285 -- =====
286 -- PATTERN 10 · DORMANT-THEN-ACTIVE
287 -- What I'm looking for: users with a 90+ day gap between consecutive
288 -- transactions followed by at least 15 transactions after
289 -- reactivation.
290 -- Expected suspects: 25-27
291 -- =====
292
293 WITH ordered_transactions AS (
294     SELECT
295         user_id,
296         txn_time,
297         LAG(txn_time) OVER (
298             PARTITION BY user_id
299             ORDER BY txn_time
300         ) AS prev_time
301     FROM transactions
302 ),
303 dormant_gaps AS (
304     SELECT
305         user_id,
306         prev_time AS dormant_from,
307         txn_time AS reactivation_time
308     FROM ordered_transactions
309     WHERE prev_time IS NOT NULL
310     AND TIMESTAMPDIFF(DAY, prev_time, txn_time) >= 90
311 ),
312 flagged_gaps AS (
313     SELECT
314         g.user_id,
315         g.dormant_from,
316         g.reactivation_time,
317         TIMESTAMPDIFF(
318             DAY,
319             g.dormant_from,
320             g.reactivation_time
321         ) AS dormant_gap_days,

```

```

321         (
322             SELECT COUNT(*)
323             FROM transactions t
324             WHERE t.user_id = g.user_id
325                   AND t.txn_time >= g.reactivation_time
326         ) AS post_gap_txn_count
327     FROM dormant_gaps g
328 )
329 SELECT
330     user_id,
331     dormant_from,
332     reactivation_time,
333     dormant_gap_days,
334     post_gap_txn_count
335 FROM flagged_gaps
336 WHERE post_gap_txn_count >= 15
337 ORDER BY post_gap_txn_count DESC;
338
339 -- My findings: 26 suspect user-gaps were flagged. Examples include user
340 -- 14526,
341 -- with a 102-day gap ending on 2024-05-20 and 55 transactions afterward,
342 -- and user 14701, with a 135-day gap ending on 2024-06-05 and 28
343 -- transactions
344 -- afterward.
345
346 -- =====
347 -- PATTERN 11 · VELOCITY SPIKE
348 -- What I'm looking for: users whose peak monthly transaction count is at
349 -- least
350 -- 5 times their average monthly count, with a peak of at least 20
351 -- transactions.
352 -- Expected suspects: 35-45
353 -- =====
354
355 WITH months AS (
356     SELECT '2024-01' AS month_key
357     UNION ALL SELECT '2024-02'
358     UNION ALL SELECT '2024-03'
359     UNION ALL SELECT '2024-04'
360     UNION ALL SELECT '2024-05'
361     UNION ALL SELECT '2024-06'
362 ),
363 monthly_counts AS (
364     SELECT
365         user_id,
366         DATE_FORMAT(txn_time, '%Y-%m') AS month_key,
367         COUNT(*) AS monthly_txn_count
368     FROM transactions
369     GROUP BY user_id, DATE_FORMAT(txn_time, '%Y-%m')
370 ),
371 user_months AS (
372     SELECT
373         u.user_id,
374         m.month_key,
375         COALESCE(mc.monthly_txn_count, 0) AS monthly_txn_count
376     FROM (
377         SELECT DISTINCT user_id

```

```

375         ) u
376     CROSS JOIN months m
377     LEFT JOIN monthly_counts mc
378         ON mc.user_id = u.user_id
379         AND mc.month_key = m.month_key
380 ),
381 user_summary AS (
382     SELECT
383         user_id,
384         AVG(monthly_txn_count) AS avg_monthly_txns,
385         MAX(monthly_txn_count) AS peak_monthly_txns,
386         SUM(
387             CASE
388                 WHEN monthly_txn_count > 0 THEN 1
389                 ELSE 0
390             END
391         ) AS active_months
392     FROM user_months
393     GROUP BY user_id
394 ),
395 final_summary AS (
396     SELECT
397         user_id,
398         avg_monthly_txns,
399         peak_monthly_txns,
400         active_months,
401         peak_monthly_txns / NULLIF(avg_monthly_txns, 0) AS
         peak_avg_ratio
402     FROM user_summary
403 )
404 SELECT
405     user_id,
406     ROUND(avg_monthly_txns, 2) AS avg_monthly_txns,
407     peak_monthly_txns,
408     active_months,
409     ROUND(peak_avg_ratio, 2) AS peak_avg_ratio
410 FROM final_summary
411 WHERE peak_monthly_txns >= 20
412     AND active_months >= 2
413     AND peak_avg_ratio > 5
414 ORDER BY peak_avg_ratio DESC;
415
416 -- My findings: 43 suspect users were flagged. Examples include user
417 14520,
418 -- whose peak month was 2024-01 with 52 transactions, user 14511,
419 -- whose
420 -- peak month was 2024-06 with 46 transactions, and user 14509, whose
421 -- peak month was 2024-02 with 36 transactions.
422
423 =====
424 -- PATTERN 12 · GEOGRAPHIC IMPOSSIBILITY
425 -- What I'm looking for: users whose consecutive transactions occur
426 -- in
427 -- different cities within 60 minutes.
428 -- Expected suspects: 15
429 --
430 =====

```

```

429     SELECT
430         user_id,
431         txn_time,
432         city,
433         LAG(txn_time) OVER (
434             PARTITION BY user_id
435             ORDER BY txn_time
436         ) AS prev_time,
437         LAG(city) OVER (
438             PARTITION BY user_id
439             ORDER BY txn_time
440         ) AS prev_city
441     FROM transactions
442 )
443 SELECT
444     user_id,
445     prev_city,
446     city AS current_city,
447     prev_time,
448     txn_time AS current_txn_time,
449     TIMESTAMPDIFF(MINUTE, prev_time, txn_time) AS gap_minutes
450 FROM ordered_transactions
451 WHERE prev_time IS NOT NULL
452     AND prev_city <> city
453     AND TIMESTAMPDIFF(MINUTE, prev_time, txn_time) <= 60
454 ORDER BY gap_minutes ASC;
455
456 -- My findings: 15 suspect users were flagged. Examples include user 14750,
457 -- moving from Visakhapatnam to Delhi in 1 minute 45 seconds on 2024-06-18,
458 -- and user 14745, moving from Surat to Thiruvananthapuram in 9 minutes
459 -- 24 seconds on 2024-05-07.
460
461 -- =====
462 -- END OF REDFLAG SUBMISSION
463 -- =====
464

```

MySQL Workbench

unloxds x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

redflag

- Tables
- Views
- Stored Procedures
- Functions

sys

Administration Schemas

Information

Schema: redflag

2nd cls* redflag_transactions transactions RedFlag_B_Harshitha (1)*

Limit to 1000 rows

```
-- PATTERN 1 - VELOCITY FRAUD
SELECT
  user_id,
  DATE(txn_time) AS attack_date,
  COUNT(*) AS daily_txn_count
FROM transactions
GROUP BY user_id, DATE(txn_time)
HAVING COUNT(*) >= 30
ORDER BY daily_txn_count DESC;
```

Result Grid

	user_id	attack_date	daily_txn_count
▶	14507	2024-05-10	50
	14530	2024-03-22	50
	14573	2024-04-01	50
	14508	2024-03-18	48
	14504	2024-01-10	44
	14503	2024-03-11	41
	14502	2024-05-31	38
	14506	2024-03-08	38
	14505	2024-03-11	37
	14501	2024-04-14	37
	14509	2024-02-06	36

Result 1 x

Output

Action Output

#	Time	Action	Message
2	07:51:40	Apply changes to red_flag	Changes applied

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Read Only Context Help Snippets

Activate Windows

Go to Settings to activate Windows.

Duration / Fetch

MySQL Workbench

unlax_ds x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

redflag

- Tables
- transactions
- Views
- Stored Procedures
- Functions

sys

Administration Schemas

Information

Schema: redflag

2nd cls* redflag_transactions transactions RedFlag_B_Harshitha (1)*

Limit to 1000 rows

```
24 SELECT
25     user_id,
26     COUNT(*) AS round_txn_count
27 FROM transactions
28 WHERE amount IN (100, 200, 500, 1000, 2000, 5000, 10000)
29 GROUP BY user_id
30 HAVING COUNT(*) >= 15
31 ORDER BY round_txn_count DESC;
32
```

Result Grid

	user_id	round_txn_count
▶	14533	30
	14534	30
	14535	30
	14532	29
	14539	29
	14541	29
	14549	29
	14548	28
	14531	27
	14555	27
	14538	26

Result 2 x

Output

Action Output

#	Time	Action	Message
14	09:50:24	SELECT user_id, DATE(pen_time) AS attack_date, COUNT(*) AS daily_txn_count ...	50 row(s) returned

Object Info Session

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Read Only Context Help Snippets

Activate Windows

Go to Settings to activate Windows

Duration / Fetch

2.313 sec / 0.000 sec

9:58 AM 10/7/2022

MySQL Workbench

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

filter objects

redflag

- Tables
- transactions
- Views
- Stored Procedures
- Functions

sys

Administration Schemas

Information

Schema: redflag

2nd cls* redflag_transactions transactions RedFlag_B_Harshitha (1)*

Limit to 1000 rows

```
44 SELECT
45     user_id,
46     DATE(txn_time) AS attack_date,
47     COUNT(*) AS tiny_txn_count
48 FROM transactions
49 WHERE amount < 10
50 GROUP BY user_id, DATE(txn_time)
51 HAVING COUNT(*) >= 30
52 ORDER BY tiny_txn_count DESC;
```

Result Grid

	user_id	attack_date	tiny_txn_count
▶	14569	2024-04-03	60
	14566	2024-05-28	60
	14564	2024-02-15	59
	14566	2024-03-15	59
	14559	2024-06-04	59
	14557	2024-03-10	58
	14567	2024-01-17	57
	14571	2024-02-19	55
	14572	2024-02-06	52
	14575	2024-03-29	52
	14573	2024-04-01	50

Result 3 x

Output

Action Output

#	Time	Action	Message
15	09:55:14	SELECT user_id, COUNT(*) AS round_txn_count FROM transactions WHERE amou...	25 row(s) returned

Object Info Session

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Read Only Context Help Snippets

Activate Windows

Go to Settings to activate Windows

Duration / Fetch

0.656 sec / 0.000 sec

9:59 AM 10/7/2024

MySQL Workbench

File Edit View Query Database Server Tools Scripting Help

Navigator

redflag_transactions transactions RedFlag_B_Harshitha (?)

Line to 1000 rows

```
73 WHERE s.user_id = f.user_id
74 AND s.status = "SUCCESS"
75 AND s.amount = f.amount
76 AND s.txn_time > f.txn_time
77 AND s.txn_time <= DATE_ADD(f.txn_time, INTERVAL 2 MINUTE)
78 )
79 GROUP BY f.user_id
80 HAVING COUNT(*) >= 10
81 ORDER BY failed_then_success_pairs DESC;
```

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Result Grid

Schema: redflag

user_id	failed_then_success_pairs
14535	35
14582	34
14576	33
14580	32
14581	32
14593	32
14590	31
14594	31
14590	31
14591	31
14585	31

Result 4

Output

Action Output

Object Info Session

16 09:00:04 SELECT user_id, DATE(txn_time) AS attack_date, COUNT(*) AS txn_count F... 20 row(s) returned

Duration / Refresh 0.141 sec / 0.000 sec

10:00 AM 10/7/2026